

ARCHITECTURE AUDIT

Marveos Operational Architecture Audit

Date: 2026-05-26

Bottom line: the platform is already usable as an operator-led onboarding and orchestration system, but it is not yet a fully self-serve production deployment platform.

The strongest implemented areas are central orchestration, hybrid authentication, commercial onboarding, workspace entitlement enforcement, and WordPress connector verification. The weakest area is real deployment execution: the system tracks deployment state more completely than it actually provisions infrastructure.

Readiness Matrix

AREA	STATUS	CURRENT REALITY
DB architecture	Yellow	Real Postgres persistence, but almost all platform state is stored as one JSONB document rather than normalized domain tables.
Auth and access	Yellow	Real native plus WordPress-bridge auth with role guards, but some protection state is process-local.
Workspace system	Yellow	Workspace creation, entitlement gating, reuse logic, and step tracking are real. Workspaces mostly represent orchestration state, not provisioned infrastructure.
Subscription system	Yellow	Plans, sessions, entitlement, and payment verification exist, but subscription truth still lives in app-managed state.
Onboarding flow	Yellow	The onboarding UI actively orchestrates real APIs, but some latent steps are bypassed and workflow logic is concentrated in one client component.
WordPress connector	Yellow	Runtime and verification endpoints are implemented, but sync hooks are still placeholders and runtime reads are broadly accessible once connector status is live enough.
Template system	Yellow	Template metadata, public filtering, and registry resync are implemented. Artifact installability is not guaranteed by a real deployment pipeline.
Deployment system	Red	Deployment links and readiness tracking exist, but infrastructure provisioning, DNS, SSL, and actual launch execution remain manual or commented future work.
Frontend architecture	Yellow	The frontend is operational, but it carries heavy orchestration responsibility that should eventually move server-side.
Infrastructure and security	Yellow/Red	Security intent is present, but rate limiting, OTP state, and some session protections are not distributed-safe.
Real client readiness today	Yellow	Viable for managed onboarding with human oversight, not yet for wide self-serve rollout.

1. DB Architecture

The operational core of the system is a Postgres-backed admin store that writes a single JSONB payload keyed by one row. In practice, this store is the platform control plane. Workspaces, commercial subscriptions, onboarding sessions, finance, operations, role configuration, templates, and support state are all merged into one object and rewritten through application-level update helpers.

This gives the team flexibility and fast iteration, but the tradeoff is clear: the database is not enforcing relational integrity, cross-entity consistency, or tenant isolation in a strong way. Those concerns are implemented in application code instead. For internal operations and early-stage platform evolution this is acceptable, but it becomes risky as throughput, concurrency, and reporting complexity increase.

2. Auth and Access

Authentication is hybrid. The app first resolves native platform sessions and then falls back to the WordPress bridge session. Role normalization maps both native roles and WordPress roles into Marveo roles, and access control is then enforced by module and action guards.

This is real authorization, not cosmetic gating. Internal surfaces are guarded through master-role permissions, and workspace-scoped APIs additionally check whether the user is internal or tied to a specific assigned workspace.

The main risk is that login attempt throttling and OTP challenge state are stored in process memory. In a single-instance deployment this works. In multi-instance production, it becomes inconsistent unless moved to shared storage.

3. Workspace System

Workspace creation is materially implemented. A workspace is created with ownership context, entitlement scope, onboarding status, rollout metadata, readiness flags, and initial missing requirements. Capacity is resolved against the selected commercial plan or account plan and can be scoped to an organization or subscription.

The real limitation is that a workspace mostly represents orchestration state. It does not prove that infrastructure, DNS, certificates, backend services, or frontend deployment artifacts exist. The workspace system is therefore operational as a service-control layer, but not yet as an infrastructure-truth layer.

4. Subscription System

Commercial onboarding supports public plans, region-aware pricing, onboarding sessions, identities, organizations, subscriptions, invoices, and payment verification. Entitlement is actively used to allow or deny workspace creation and onboarding continuation.

This is sufficient for a managed commercial funnel, including trial expiry handling and client-specific workspace limits. The weakness is architectural rather than functional: subscription truth is still maintained inside the same application store instead of being anchored to a hardened billing system with independent lifecycle ownership.

5. Onboarding Flow

The onboarding UI is a real operational workflow. It restores draft state from local storage, hydrates from onboarding sessions, checks entitlement, loads publishable templates, verifies connector access, creates the workspace, pushes onboarding step updates, triggers profile provisioning, optionally assigns support, and loads a launch checklist.

One notable implementation detail is that the initial wizard only truly exposes the plan, profile, review, deploying, and ready path. The file still contains website type and details views, but the current guard logic prevents those from being part of the initial route and pushes the user back into the shorter path.

That means the live onboarding product is narrower than the component suggests. It is implemented, but it has not yet converged into a clean single workflow model.

6. WordPress Connector

The connector is good enough to support probing, runtime content reads, deployment-status reads, module activation, content mapping, and initial admin bootstrapping. Marveos uses it for connector preflight and workspace verification, and stores resulting connector state and metadata back in the admin store.

This is materially useful. Existing WordPress onboarding is one of the stronger parts of the system.

However, the connector is not yet a mature sync engine. User, order, and product hooks remain explicit placeholders. Also, many runtime read endpoints are allowed whenever connector status is pending setup or active, rather than being tied to per-request authenticated users.

7. Template System

Templates are managed as operational metadata with publish rules. Public templates must be active, public, and either have a confirmed artifact or explicitly require support. The platform can also resync template definitions from the sibling marveo-templates repository.

This means the template catalog is real and curated. It is not a mock.

The constraint is that template availability does not equal automated deployment readiness. The platform knows which template is allowed for which market and plan, but that does not yet mean the system can fully provision and launch that template without operator assistance.

8. Deployment System

This is the least production-ready subsystem. Deployment links exist, can be listed, can be finalized, and do update workspace state. Launch guard and launch checklist logic also exist.

But the route that finalizes deployment links explicitly documents the important production steps as future work: infrastructure provisioning, DNS configuration, SSL certificate generation, webhook registration, and content sync triggering. In other words, the platform currently tracks deployment state much better than it actually performs deployment.

The deployment system today is a workflow coordinator and readiness evaluator, not a complete automated deployment engine.

9. Frontend Architecture

The onboarding frontend is a large client-side controller. It is not just rendering forms. It manages workflow transitions, persistence, entitlement reactions, connector probing, workspace creation, support assignment, and readiness display.

This is effective for shipping product quickly, but it also means critical orchestration logic is concentrated in a single client component. Over time that will make maintenance, testability, and workflow correctness harder, especially as more onboarding modes are added.

10. Infrastructure and Security

The codebase shows clear intent toward hardened operations: Postgres persistence, audit logs, rate limiting on public endpoints, session security defaults, login protection settings, OTP

challenge flow, and permission-based API access.

The current weakness is not lack of security thinking. It is operational maturity. Rate limits are in memory. OTP challenges are in memory. Login attempt tracking is in memory. Some connector runtime reads are loosely protected. These choices are common in early-stage systems, but they are not the final form of a multi-instance production platform.

11. Client Deployment Readiness Today

If a real client were onboarded today, the platform could support a managed-service motion: capture plan, profile, and entitlement state, create the workspace record, verify or fail an existing WordPress connector flow, assign support, and generate launch-checklist and readiness views.

What it cannot yet do reliably end to end is self-serve production deployment for new websites without human intervention. New website provisioning especially still depends on follow-through outside the implemented deployment engine.

The honest classification is: operator-led client onboarding is feasible; self-serve production deployment at scale is not yet ready.

12. Final Recommendation and Rollout Strategy

The right strategy is to treat the current platform as a managed onboarding and orchestration OS first.

1. Use the current system for internal-team-led onboarding and controlled client pilots.
2. Do not market the current deployment path as fully automated self-serve provisioning.
3. Prioritize replacing process-local login, OTP, and rate-limit state with shared storage.
4. Build a real deployment executor behind the current launch and deployment-link surfaces.
5. Tighten connector runtime authorization and formalize command authentication and auditability.
6. Move workflow-critical orchestration out of the large client-side onboarding component and into explicit backend workflow handlers.

Production readiness verdict: suitable for controlled managed onboarding, not yet suitable for broad self-serve production rollout.

The platform already has meaningful operational depth. The missing work is not conceptual. It is concentrated in infrastructure execution, distributed security hardening, and converting tracked deployment state into real automated deployment outcomes.

Prepared from the current code implementation in marveos, marveo-connector, and marveo-templates. This report describes the observed system as implemented, not an ideal target architecture.